

Tut 12

Anne

Jakarta EE

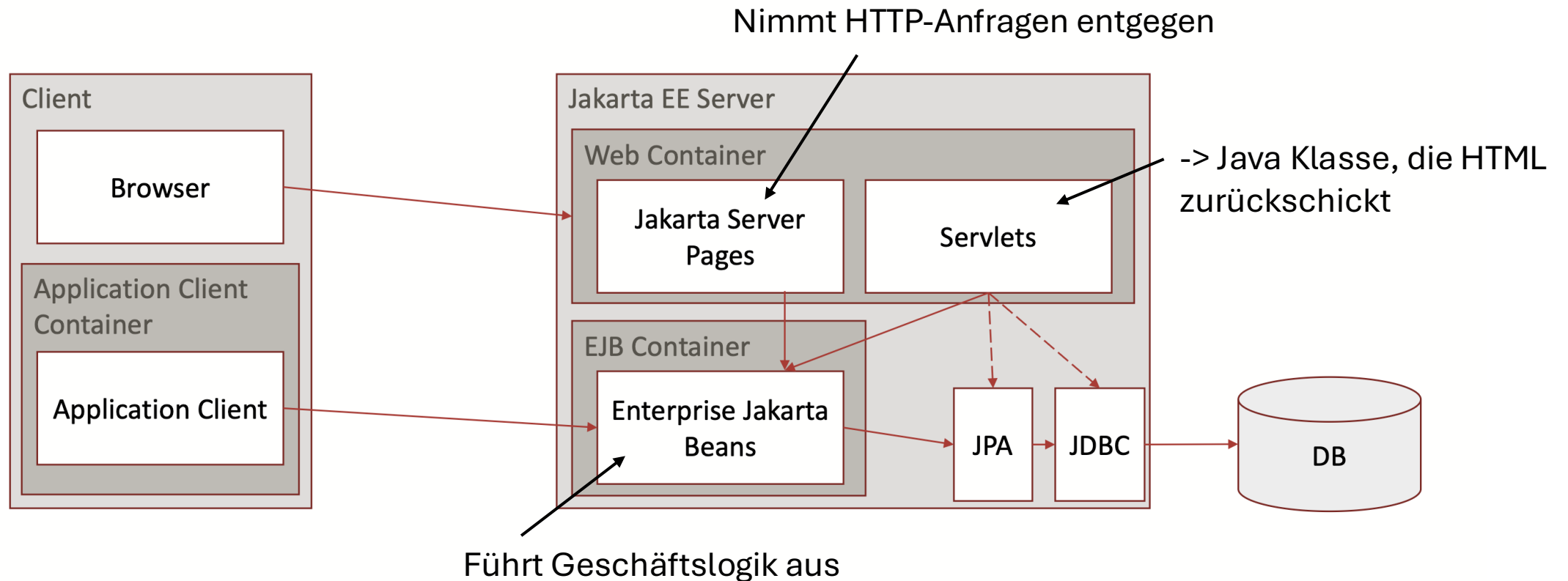
- Wir wollen Webanwendung bauen
- -> immer das gleiche:
- Sie muss auf HTTP-Anfragen aus dem Browser reagieren
- Sie muss Objekte erzeugen und verwalten
- Sie muss mit einer Datenbank sprechen
- Sie muss Nutzer einloggen/absichern
- Sie muss mit vielen Nutzern gleichzeitig klarkommen (Concurrency)....

- -> müssen wir nicht Selbst programmieren
- Jakarta EE
 - Middleware, die diese Laufzeitfunktionen bereitstellt
 - Sammlung von Standards/Spezifikationen, die genau das regeln

Jakarta EE

In Container aufgeteilt -> isolierte Zonen die Aufgaben übernehmen

Jakarta EE overview



Aufgabe 3 – Webanwendungen

Sie erhalten den Auftrag, für die Berliner Verkehrsbetriebe (BVG) ein internetfähiges Fahrkartenreservierungssystem zu konzipieren und später auch zu implementieren, über das der Kauf von Fahrscheinen und eine Fahrplanauskunft möglich sein soll. Bei Fahrscheinkäufen müssen User E-Mail-Adresse, Kundennummer und Kontoverbindung angeben, bei Fahrplanauskünften die Start- und Zielhaltestelle sowie eine Uhrzeit und ein Datum. In Ihrer Projektskizze haben Sie der BVG dargelegt, dass Sie Technologien wie JavaScript und Jakarta-EE-Web-Container einsetzen wollen.

Was sind hier Vor- und Nachteile von JavaScript und Jakarta EE?

Clientseitig (JavaScript)

Vorteile

- Sofortiges Feedback bei falscher Eingabe (z. B. ungültiges E-Mail-Format)
- Entlastet den Server -> fehlerhafte Anfragen werden gar nicht erst gesendet

Nachteile / Grenzen

- Code ist durch Nutzer manipulierbar oder per Add-on blockierbar -> Prüfung ist nicht verbindlich
- Muss von allen Browsern unterstützt werden
- Manche Prüfungen unmöglich (z. B. Kundennummer gegen DB verifizieren)

Serverseitig (Jakarta EE)

Vorteile

- Verbindlich & sicher -> kann von Nutzern nicht umgangen werden
- Voller Zugriff auf DB & Geschäftslogik (Kundennummer, Kontodaten prüfen)
- Server kontrolliert Java-Version/Umgebung selbst

Nachteile

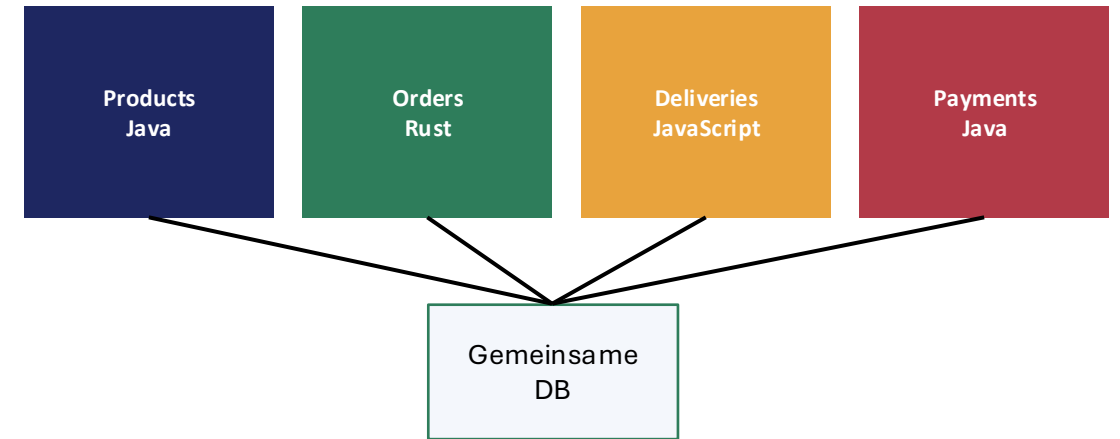
- Feedback erst nach Roundtrip zum Server -> langsamer für den Nutzer
- Jede Anfrage belastet den Server, auch offensichtlich fehlerhafte

Fazit: Kombination beider Techniken sinnvoll

Microservices



SOA



Gemeinsamkeit: Baue deine Anwendung nicht als einen großen Monolithen, sondern als mehrere kleine, unabhängige Bausteine (Services), die miteinander kommunizieren

- **Microservices schneiden nach Business-Fähigkeit**
- Ein Service = eine komplette Geschäftsfunktion, von oben bis unten (full-stack)
- Choreography

- **SOA schneidet nach Technik**
- Ein Service = eine technische Funktionalität, die möglichst oft wiederverwendet werden soll
- Orchestrierung

FaaS - serverless

- Container/Infrastruktur bereitstellen
 - wenn deine Funktion aufgerufen wird, startet die Plattform automatisch einen Container dafür
- Skalierung
 - kommen 1000 Anfragen gleichzeitig, startet sie einfach 1000 Container-Instanzen parallel
- Scale-to-Zero
 - kommt gar keine Anfrage, läuft auch nichts, und du zahlst nichts
- Failure Handling, Lifecycle-Management
 - alles, worum du dich bei einem eigenen Server kümmern müsstest
- Funktionen sind event driven
- Nachteil: Wechsel extrem aufwändig

Aufgabe 4 – Serverless

Sie haben die Aufgabe, ein Anwendungssystem zu entwickeln. In dieser Aufgabe ist der konkrete Use-Case relativ egal. Das System kann unter folgenden unterschiedlichen Lasten stehen: konstante Last (z. B. immer 100 req/s), unregelmäßige kleine Last (z. B. 30 Tage nichts, 1 Tag 5 req/s), unregelmäßig große Last (z. B. 30 Tage nichts, 1 Tag 100 req/s).

Sie bekommen die Aufgabe, das System auf einen der drei Parameter zu optimieren: Laufzeitkosten minimieren, Entwicklungszeit minimieren, Latenz minimieren.

Entscheiden Sie sich für jeden Fall für eine allgemeine Softwarearchitektur und begründen Sie Ihre Entscheidung: Monolith, FaaS, SOA.

	Laufzeitkosten min.	Entwicklungszeit min.	Latenz min.
Konstante Last	Monolith: kann genau auf die Last ausgerichtet werden	FaaS: einfach Code schreiben geht am schnellsten	Monolith: tiefe Integration erlaubt „Abkürzungen“ für performanten Code
Oft 0 Last, selten kleine Last	FaaS: <i>scale-to-zero</i> verursacht keine Kosten bei 0 Last	FaaS: s. o.	Monolith: s. o.
Oft 0 Last, selten sehr hohe Last	FaaS oder SOA: Monolith ist evtl. schlecht skalierbar, SOA und FaaS skalieren besser	FaaS: s. o.	Monolith: s. o.

Klausurähnliche Aufgaben:

- a) Erläutern Sie kurz die Rolle von Topics im Publish/Subscribe-Messaging. Wofür werden diese benötigt?
- b) Stellen Sie sich ein Pub/Sub-System vor, in dem tausende Sensoren in einem Windpark Messwerte an mehrere Auswertungsdienste senden, wobei die Netzwerkverbindung der Sensoren häufig instabil ist. Ist in diesem Szenario ein Broker-Based oder ein Brokerless Pub/Sub-System sinnvoller? Erläutern Sie Ihre Antwort in ca. zwei Sätzen. Hinweis: Sie bekommen nur Punkte für die Kombination aus Antwort und Begründung.

c) Geben Sie für folgende Anwendungsszenarien an, welche Art der Kommunikation am besten geeignet ist. Mögliche Kommunikationsarten: Synchroner Kommunikation, Message Queues, Pub/Sub, oder eine Kombination dieser. Begründen Sie kurz in einem Satz.

1. Ein Wetterdienst verteilt Wetter-Updates unterschiedlicher Regionen an beliebig viele interessierte Wetter-Apps.

Pub/Sub

2. Eine Bank führt eine Online-Überweisung durch und muss dem Kunden sofort mitteilen, ob die Transaktion erfolgreich war.

Synchroner Kommunikation

3. Ein Bewegungsmelder in einem Smart Home schaltet eine einzelne Lampe an oder aus.

Synchroner Kommunikation

4. Eine Video-Plattform verarbeitet hochgeladene Videos in mehreren Schritten nacheinander: Transkodierung, Thumbnail-Erzeugung, Benachrichtigung des Uploaders.

Message Queues

Aufgabe 3 – XML & XML-Schema (10 Punkte)

Gegeben seien die folgenden vier XML-Schemafragmente und fünf XML-Codefragmente. Bestimmen Sie für alle Kombinationen, ob das jeweilige Codefragment erfolgreich gegen das Schemafragment validiert werden kann. Tragen Sie ✓ (validiert) oder ✗ (validiert nicht) in die Tabelle ein.

XSD Fragment 1:

```
<xs:element name="task" type="priority"/>
<xs:simpleType name="priority" type="xs:string"/>
```

XSD Fragment 2:

```
<xs:element name="task" type="priority"/>
<xs:simpleType name="priority">
  <xs:restriction base="xs:string">
    <xs:maxLength value="4"/>
  </xs:restriction>
</xs:simpleType>
```

XSD Fragment 3:

```
<xs:element name="task" type="priority"/>
<xs:simpleType name="priority">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Low"/>
    <xs:enumeration value="Medium"/>
    <xs:enumeration value="High"/>
    <xs:enumeration value="Critical"/>
  </xs:restriction>
</xs:simpleType>
```

XSD Fragment 4:

```
<xs:element name="task" type="priority"/>
<xs:complexType name="priority">
  <xs:choice>
    <xs:element name="value" type="xs:string" maxOccurs="1"/>
    <xs:element name="value2" type="xs:integer" maxOccurs="1"/>
  </xs:choice>
</xs:complexType>
```

- XML Fragment A: <task>High</task>
- XML Fragment B: <ticket>High</ticket>
- XML Fragment C: <task>Med</task>
- XML Fragment D:
 <task>
 <value>Urgent</value>
 </task>
- XML Fragment E: <task>Low</task>

	Schema 1	Schema 2	Schema 3	Schema 4
A <task>High</task>				✗
B <ticket>High</ticket>	✗	✗	✗	✗
C <task>Med</task>			✗	✗
D <task><value>Urgent</value></task>	✗	✗	✗	
E <task>Low</task>				✗

ACID

- **Atomicity:** Eine Transaktion wird entweder komplett ausgeführt oder überhaupt nicht.
- **Consistency:** Die Datenbank wechselt nur von einem gültigen Zustand in den nächsten.
- **Isolation:** Gleichzeitige Transaktionen beeinflussen sich nicht und laufen wie nacheinander ab.
- **Durability:** Einmal gespeicherte Daten gehen auch bei einem Systemabsturz nicht mehr verloren.

a) Geben Sie die SQL-Query an, die die Spalten NAME und EMAIL aus der Tabelle EMPLOYEES zurückgibt, für deren Zeilen der Wert DEPT_ID gleich 7 ist.

SELECT NAME, EMAIL FROM EMPLOYEES WHERE DEPT_ID = 7;

b) Welche Kategorie von HTTP-Statuscodes wird jeweils vom Server zurückgegeben? Setzen Sie pro Zeile genau einen Haken.

Szenario	2xx	3xx	4xx	5xx
Eine neue Ausleihe konnte erfolgreich angelegt werden.	✓			
Ein Client sendet ein Buch-Update im falschen Datenformat.			✓	
Der Server hat interne Probleme bei der Bearbeitung der Anfrage.				✓
Der Client fragt ein Buch mit einer ID ab, die nicht existiert.			✓	
Die angefragte Ressource ist dauerhaft unter einer neuen URL erreichbar.		✓		

c) Welche der ACID-Eigenschaften ist für Datenbanktransaktionen bei folgenden Szenarien besonders wichtig? Setzen Sie pro Zeile genau einen Haken.

Szenario	A	C	I	D
Ein neu angelegtes Konto darf keinen negativen Kontostand haben.		✓		
Nach Abschluss einer Überweisung bleibt das Ergebnis auch nach einem Stromausfall gespeichert.				✓
Mehrere Nutzer können gleichzeitig Tickets für ein Konzert buchen, ohne dass sich die Buchungen gegenseitig verfälschen.			✓	
Eine Flugbuchung mit Hinflug, Rückflug und Hotel wird nur komplett oder gar nicht durchgeführt.	✓			
Studierende können sich nur für eine Prüfung anmelden, wenn sie im zugehörigen Modul eingeschrieben sind.		✓		
Eine fehlgeschlagene Transaktion beeinflusst keine andere gleichzeitig laufende Transaktion auf denselben Zellen.			✓	

```

<description xmlns="http://www.w3.org/ns/wSDL"
xmlns:tns="http://example.com/library/2024/"
targetNamespace="http://example.com/library/2024/">
<types>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://example.com/library/2024/">

<xsd:complexType name="GetBookInfoType">
<xsd:sequence>
<xsd:element name="isbn" type="xsd:string"/>
</xsd:sequence>
</xsd:complexType>

<xsd:complexType name="GetBookInfoResponseType">
<xsd:sequence>
<xsd:element name="Title" type="xsd:string"/>
<xsd:element name="Author" type="xsd:string"/>
<xsd:element name="Year" type="xsd:int"/>
<xsd:element name="Price" type="xsd:float"/>
</xsd:sequence>
</xsd:complexType>

<xsd:element name="GetBookInfo" type="tns:GetBookInfoType"/>
<xsd:element name="GetBookInfoResponse" type="tns:GetBookInfoResponseType"/>
</xsd:schema>
</types>

```

```

<interface name="LibraryService">
<operation name="GetBookInfo">
<input message="tns:GetBookInfo"/>
<output message="tns:GetBookInfoResponse"/>
</operation>
</interface>
... (Binding und Service-Definition analog SOAP/HTTP) ...
</description>

```

Aufgabe: Schreiben Sie die äquivalente .proto-Datei mit service, rpc-Methode und den passenden message-Typen.

```

service LibraryService {
  rpc GetBookInfo (GetBookInfoRequest) returns
  (GetBookInfoResponse);
}

message GetBookInfoRequest {
  string isbn = 1;
}

message GetBookInfoResponse {
  string title = 1;
  string author = 2;
  int32 year = 3;
  float price = 4;
}

```

GitHUb –
Code kommt
später

